# LAB: GPIO Digital InOut 7-segment

**Date:** 2024-09-22

**Author/Partner:** HWANG MooYeol

**Github:** repository link

**Demo Video:** Problem1 Demo video ,Problem2 Demo video

**PDF version:**LAB_GPIO_InOut_7segment PDF

# Introduction

In this lab, you are required to create a simple program to control a 7-segment display to show a decimal number (0~9) that increases by pressing a push-button.

You must submit

- LAB Report (*.pdf)
- Zip source files(LAB***.c, ecRCC2.h, ecGPIO2.h etc...).
    - Only the source files. Do not submit project files

## Requirement

### Hardware

- MCU
    - NUCLEO-F411RE
- Actuator/Sensor/Others:
    - 7-segment display(5101ASR)
    - Array resistor (330 ohm)
    - decoder chip(74LS47)
    - breadboard

### Software

- Keil uVision, CMSIS, EC_HAL library

# Exercise

Fill in the table

| Port/Pin | Description | Register setting |
|---|---|---|
| Port A Pin 5 | Clear Pin5 mode | GPIOA->MODER &= ~(3<<(5*2)) |
| Port A Pin 5 | Set Pin5 mode = Output | GPIOA->MODER \|= 1<<(5*2) |
| Port A Pin 6 | Clear Pin6 mode | GPIOA->MODER &= ~(3<<(6*2)) |
| Port A Pin 6 | Set Pin6 mode = Output | GPIOA->MODER \|= 1<<(6*2) |
| Port A Pin Y | Clear PinY mode | GPIOA->MODER &=~ (3<<(Y*2)) |
| Port A Pin Y | Set PinY mode = Output | GPIOA->MODER \|= 1<<(Y*2) |
| Port A Pin 5~9 | Clear Pin5~9 mode | GPIOA->MODER &= ~(1023<<(5*2)) |
| | Set Pin5~9 mode = Output | GPIOA->MODER \|= 341<<(5*2) |
| Port X Pin Y | Clear Pin Y mode | GPIOX->MODER &=~(3<<(Y*2)) |
| | Set Pin Y mode = Output | GPIOX->MODER \|= 1<<(Y*2) |
| Port A Pin5 | Set Pin5 otype=push-pull | GPIOA->OTYPER = 0<<5 |
| Port A PinY | Set PinY otype=push-pull | GPIOA-> OTYPER = 0<<Y |
| Port A Pin5 | Set Pin5 ospeed=Fast | GPIOA->OSPEEDR = 2<<(5*2) |
| Port A PinY | Set PinY ospeed=Fast | GPIOA-> OSPEEDR =2<<(Y*2) |
| Port A Pin 5 | Set Pin5 PUPD=no pullup/down | GPIOA->OTYPER = 0<<(5*2) |
| Port A Pin Y | Set PinY PUPD=no pullup/down | GPIOA-> OTYPER =0<<(Y*2) |

---

# Problem 0: Connection of 7-Segment Display and Decoder

## Procedure

Review 7-segment Decoder and Display from Digital Logic lecture.

- Read here: 7-segment tutorial
- Read here: How to connect 7-segment decoder to MCU

### 1. 7-segment display connection

First, connect the common anode 7-segment display with the given array resistors.

Then, apply VCC and GND to the 7-segment display.

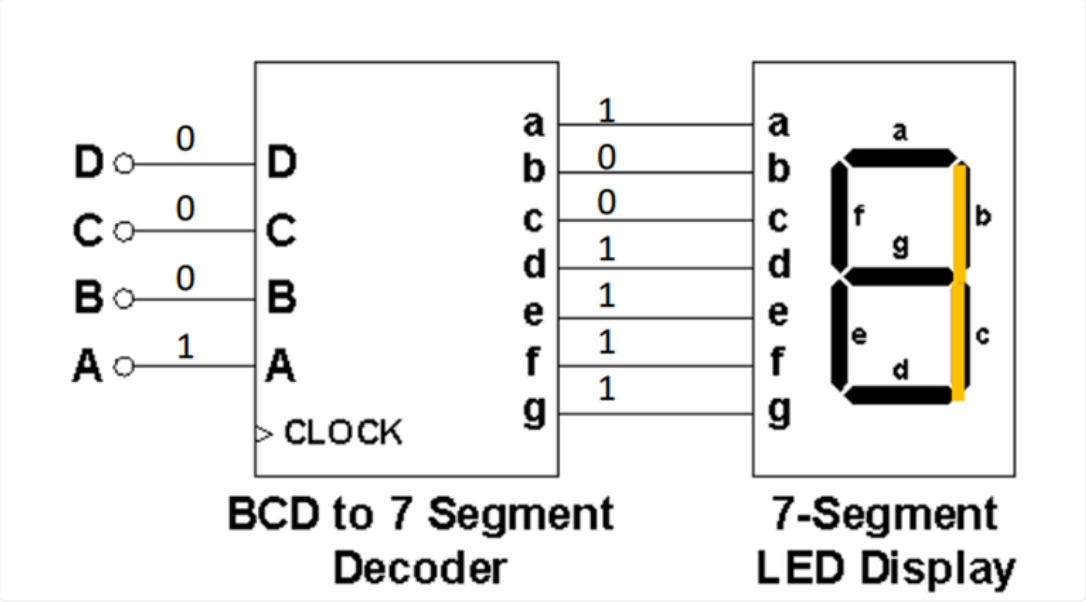
Check that all LEDs of 7-segment work properly

- Give 'H' signal to each 7-segment pin of 'a'~'g' . Observe if that LED is turned ON or OFF
- Example: Connect VCC to all 'a'~'g' pins

## 2. BCD 7-segment decoder connection

The popular BCD 7-segment decoder chip is **74LS47.** With the BCD chip, you need only 4 DOUT pins of MCU to display from 0 to 9.

Connect the decoder chip (**74LS47**) on the bread board.

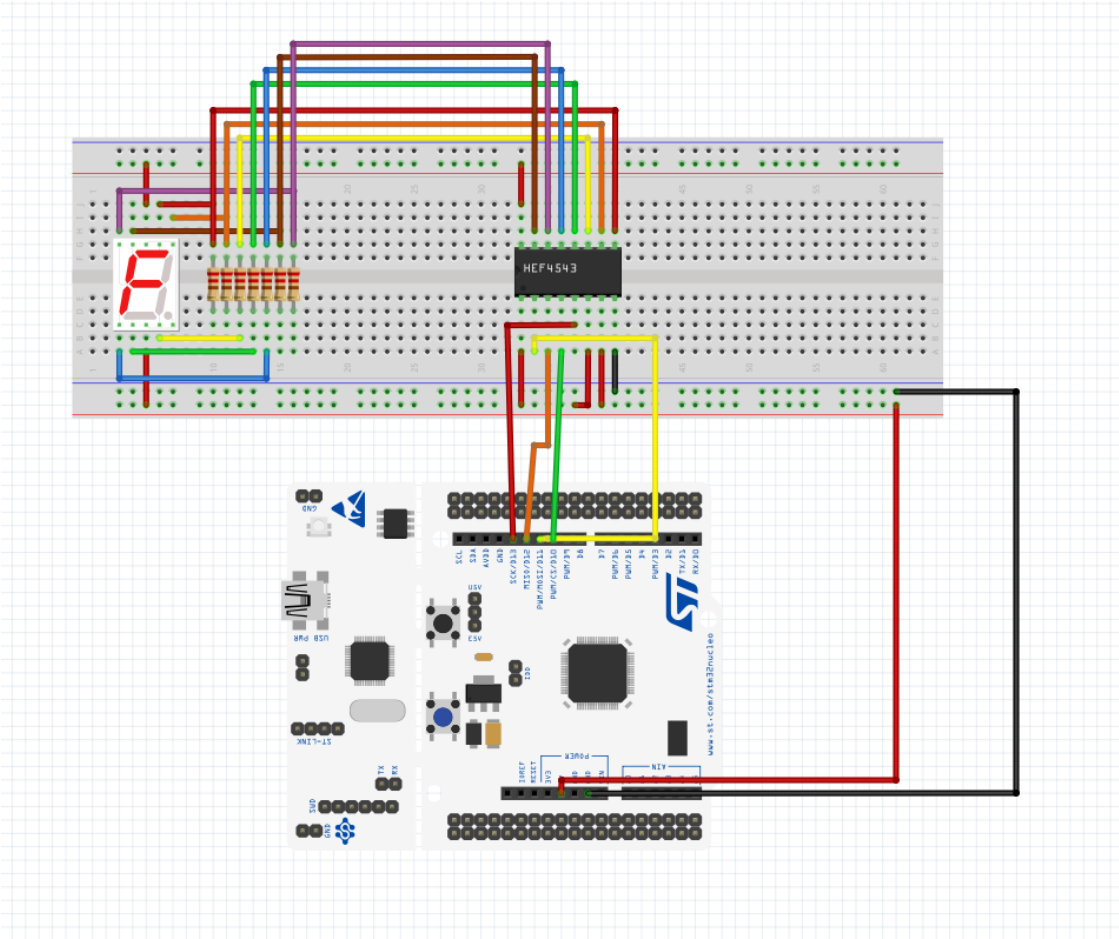



Then, Check that the decoder chip works properly

- Supply a combination of VCC/GND to the pins of 'A'~'D' of the decoder.
- Check if the 7-segment LED display shows the correct number

## Connection Diagram

Circuit diagram

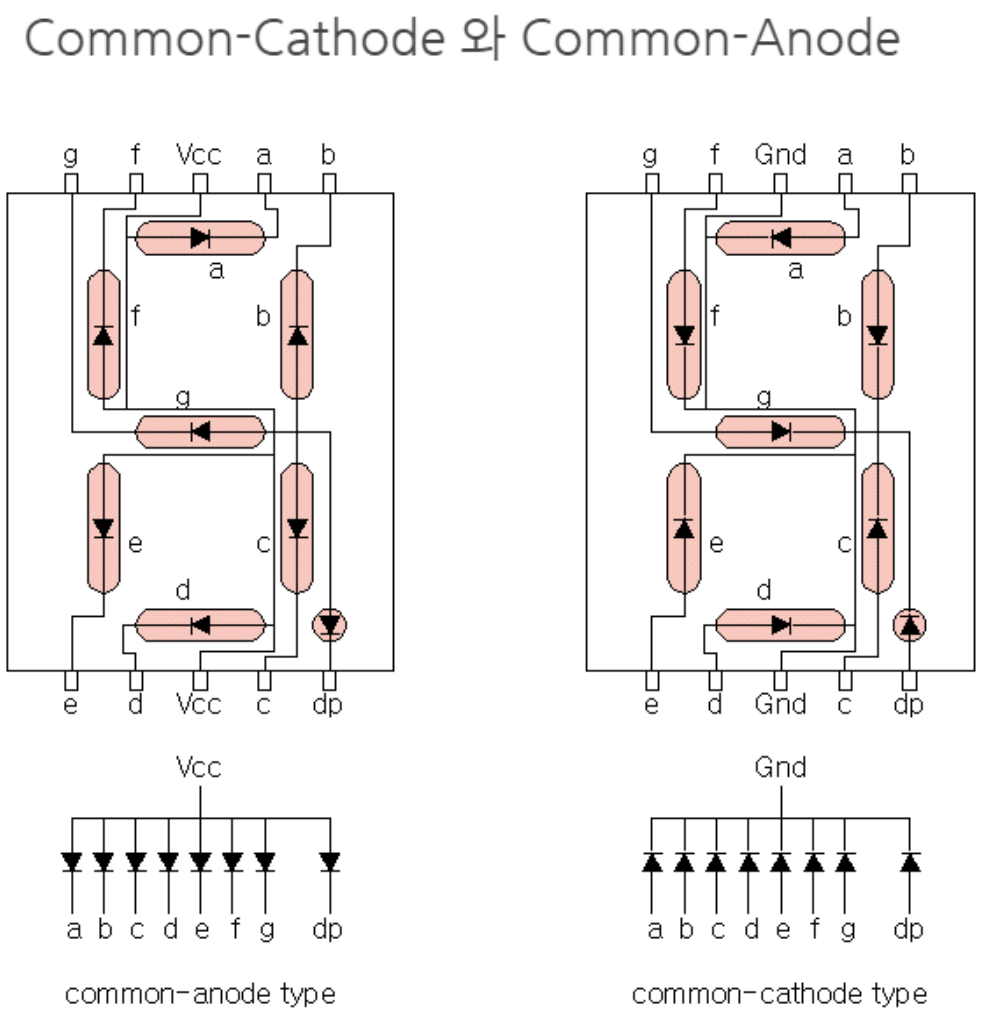



## Discussion

1. Draw the truth table for the BCD 7-segment decoder with the 4-bit input.

| A | B | C | D | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | x | x | x | x | x | x | x |
| 1 | 0 | 1 | 1 | x | x | x | x | x | x | x |
| 1 | 1 | 0 | 0 | x | x | x | x | x | x | x |
| 1 | 1 | 0 | 1 | x | x | x | x | x | x | x |
| 1 | 1 | 1 | 0 | x | x | x | x | x | x | x |
| 1 | 1 | 1 | 1 | x | x | x | x | x | x | $\boxed{x}$ |

2. What are the common cathode and common anode of 7-segment display?




Common-Anode type LED, the anodes (+) of the LEDs are tied together. The tied lead is connected to Vcc (5V), and each LED's cathode (-) is connected to either Vcc or GND (0V) to determine whether the light is turned on or off. Only the LEDs connected to the GND terminal will light up,

On the other hand, the Common-Cathode type LED ties the cathodes (-) of the LEDs together and connects them to GND, and whether to connect each LED's anode (+) to GND or Vcc determines whether the LED lights up

3. Does the LED of a 7-segment display (common anode) pin turn ON when 'HIGH' is given to the LED pin from the MCU?

Since one of Common-Anode type LEDs are all tied at Vcc, 7-segment display pins turns ON when **'LOW'** is given to the LED pin from the MCU.

---

## Problem 1: Display a Number with Button Press

## Procedure

1. Create a new project under the directory `\repos\EC\LAB\LAB_GPIO_7segment`

- The project name is "**LAB_GPIO_7segment".**
- Create a new source file named as "**LAB_GPIO_7segment.c"**
- Refer to the [sample code]

> You MUST write your name on the source file inside the comment section.

2. Include your updated library in `\repos\EC\lib\` to your project.

- **ecGPIO.h, ecGPIO.c**
- **ecRCC.h, ecRCC.c**

3. Declare and Define the following functions in your library

```
void sevensegment_display_init(PinNames_t pinNameA, PinNames_t pinNameB,
PinNames_t pinNameC, PinNames_t pinNameD);

PinName_t pinName1, pinName2, pinName3, pinName4;

void sevensegment_display_init(PinName_t pinNameA, PinName_t pinNameB, PinName_t
pinNameC, PinName_t pinNameD) {
    pinName1 = pinNameA;
    pinName2 = pinNameB;
    pinName3 = pinNameC;
    pinName4 = pinNameD;

    GPIO_init(pinName1, OUTPUT);
    GPIO_init(pinName2, OUTPUT);
    GPIO_init(pinName3, OUTPUT);
    GPIO_init(pinName4, OUTPUT);
}

void sevensegment_display(uint8_t  num);

void sevensegment_display(uint8_t num) {
    uint8_t pattern_map[10] = {
        0b0000, // 0
        0b0001, // 1
        0b0010, // 2
        0b0011, // 3
        0b0100, // 4
        0b0101, // 5
        0b0110, // 6
        0b0111, // 7
        0b1000, // 8
        0b1001  // 9
    };

    uint8_t pattern = pattern_map[num];

    GPIO_write(pinName1, (pattern & 0b0001) ? HIGH : LOW);
    GPIO_write(pinName2, (pattern & 0b0010) ? HIGH : LOW);
    GPIO_write(pinName3, (pattern & 0b0100) ? HIGH : LOW);
    GPIO_write(pinName4, (pattern & 0b1000) ? HIGH : LOW);
```

```
}
```

- num: 0 to 9 only (unsigned)

4. Configure and connect the MCU to the circuit
5. First, check that every number, 0 to 9, can be displayed properly
6. Then, create a code that increase the displayed number from 0 to 9 with each button press.

- After the number '9', it should start from '0' again.

## Configuration

Configure the MCU

| Digital In for Button (B1) | Digital Out for 7-Segment |
|---|---|
| Digital In | Digital Out |
| PC13 | PA7, PB6, PC7, PA9 |
| PULL-UP | Push-Pull, No Pull-up-Pull-down, Medium Speed |

### Code

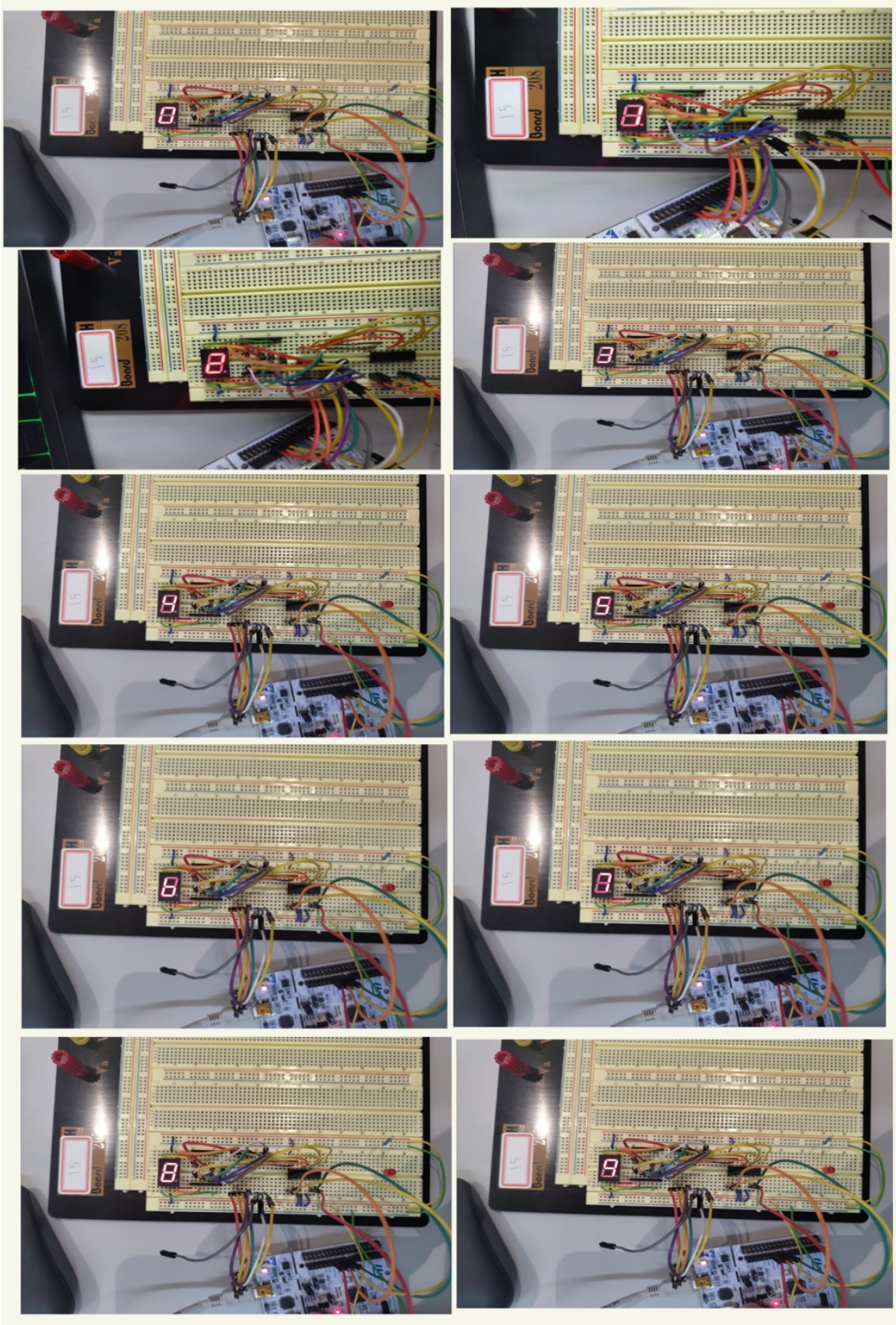
Your code goes here: [ADD Code LINK such as github]

```
#include "stm32f4xx.h"
#include "ecGPIO2_student.h"
#include "ecRCC2.h"

#define BUTTON_PIN PC_13

void setup(void);

int main(void) {
    // Initialiization -------------------------------------------------------
    setup();
    unsigned int cnt = 0;
    int lastButtonState = HIGH;

    // Inifinite Loop ---------------------------------------------------------
    while(1){

        int buttonState = GPIO_read(BUTTON_PIN);
        sevensegment_display(cnt % 10);
        if(buttonState == LOW && lastButtonState == HIGH) {
            cnt++;
            if (cnt > 9) cnt = 0;
        }
        lastButtonState = buttonState;
    }
}


// Initialiization
void setup(void)
{
```

```
    RCC_HSI_init();
    GPIO_init(BUTTON_PIN, INPUT);  // calls RCC_GPIOC_enable()
    GPIO_pupd(BUTTON_PIN, PULL_UP);

    sevensegment_display_init(PA_7, PB_6, PC_7, PA_9);  // Decoder input A,B,C,D
    GPIO_otype(PA_7, PUSH_PULL);
    GPIO_otype(PB_6, PUSH_PULL);
    GPIO_otype(PC_7, PUSH_PULL);
    GPIO_otype(PA_9, PUSH_PULL);

    GPIO_pupd(PA_7, NO_PUPD);
    GPIO_pupd(PB_6, NO_PUPD);
    GPIO_pupd(PC_7, NO_PUPD);
    GPIO_pupd(PA_9, NO_PUPD);

    GPIO_ospeed(PA_7, MEDIUM_SPEED);
    GPIO_ospeed(PB_6, MEDIUM_SPEED);
    GPIO_ospeed(PC_7, MEDIUM_SPEED);
    GPIO_ospeed(PA_9, MEDIUM_SPEED);
}
```

## Results

Experiment images and results



demo video link

## Discussion

Through this problem,  learned how to operate the decoder with four pins of the MCU.

There were 9 output values, so the code got longer when the if and switch statements were used. To solve this problem, it was possible to shorten the code by setting the input bits in an array.

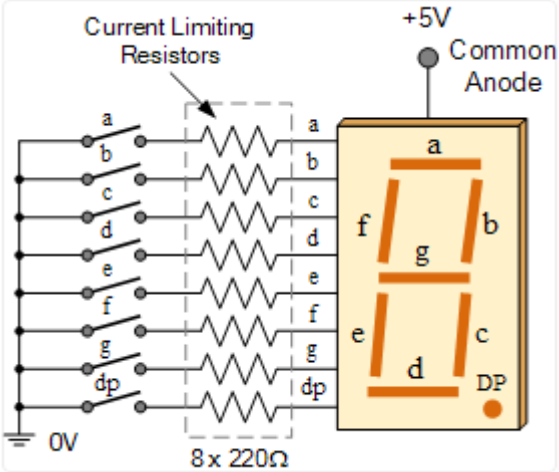
In addition, pin information could be retrieved from the init function, but pin information could not be retrieved from the display function. This was solved by using global variables.

---

# Problem 2: Program BCD-7-segment decoder

Instead of using the decoder chip, we are going to make the 7-segment decoder with the MCU programming.

> Do not use the 7-segment decoder for this problem




## Procedure

1. Use the same project and source file.
2. Include your updated library in `\repos\EC\lib\` to your project.

- **ecGPIO.h, ecGPIO.c**
- **ecRCC.h, ecRCC.c**

3. Declare and Define the following functions in your library

```
void sevensegment_decoder_init(void);

void sevensegment_decoder_init(void){
    GPIO_init(PA_5, OUTPUT);  // 7segment-a
    GPIO_init(PA_6, OUTPUT);  // 7segment-b
    GPIO_init(PA_7, OUTPUT);  // 7segment-c
    GPIO_init(PB_6, OUTPUT);  // 7segment-d
    GPIO_init(PC_7, OUTPUT);  // 7segment-e
    GPIO_init(PA_9, OUTPUT);  // 7segment-f
    GPIO_init(PA_8, OUTPUT);  // 7segment-g
    GPIO_init(PB_10, OUTPUT); // 7segment-dp
}

void sevensegment_decoder(uint8_t  num);

void sevensegment_decoder(uint8_t num) {
    uint8_t segment_bit[10] = {
        0b00111111, // 0: a, b, c, d, e, f
        0b00000110, // 1: b, c
        0b01011011, // 2: a, b, d, e, g
        0b01001111, // 3: a, b, c, d, g
        0b01100110, // 4: b, c, f, g
```

```
        0b01101101, // 5: a, c, d, f, g
        0b01111101, // 6: a, c, d, e, f, g
        0b00000111, // 7: a, b, c
        0b01111111, // 8: a, b, c, d, e, f, g
        0b01101111  // 9: a, b, c, d, f, g
    };

    uint8_t pattern = segment_bit[num];

    GPIO_write(PA_5, (pattern & 0b00000001) ? LOW : HIGH); // a
    GPIO_write(PA_6, (pattern & 0b00000010) ? LOW : HIGH); // b
    GPIO_write(PA_7, (pattern & 0b00000100) ? LOW : HIGH); // c
    GPIO_write(PB_6, (pattern & 0b00001000) ? LOW : HIGH); // d
    GPIO_write(PC_7, (pattern & 0b00010000) ? LOW : HIGH); // e
    GPIO_write(PA_9, (pattern & 0b00100000) ? LOW : HIGH); // f
    GPIO_write(PA_8, (pattern & 0b01000000) ? LOW : HIGH); // g
    GPIO_write(PB_10, 1);

}
```

- num: 0 to 9 only (unsigned)

4. Configure and connect the MCU to the circuit
5. First, check that every number, 0 to 9, can be displayed properly
6. Then, create a code that increases the displayed number from 0 to 9 with each button press.

- After the number '9', it should start from '0' again.

## Configuration

Configure the MCU

| Digital In for Button (B1) | Digital Out for 7-Segment |
|---|---|
| Digital In | Digital Out |
| PC13 | PA5, PA6, PA7, PB6, PC7, PA9, PA8, PB10 ('a'~'h', respectively) |
| PULL-UP | Push-Pull, No Pull-up-Pull-down, Medium Speed |

## Code

Your code goes here: [ADD Code LINK such as github]

- [Example Code for MCU configuration]
- [Example code of 7-segment decoder control]

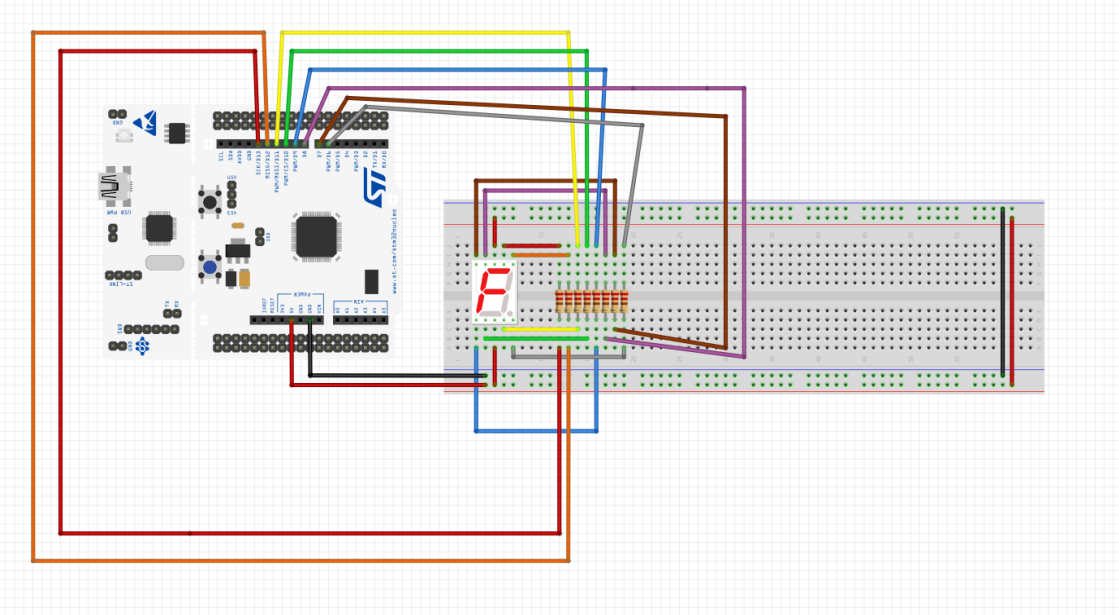
> Explain your source code with the necessary comments.

```
#include "stm32f4xx.h"
#include "ecRCC2.h"
#include "ecGPIO2_student.h"

void setup(void);

int main(void) {
    // Initialiization ---------------------------------------------------------
```

```
    setup();
    unsigned int cnt = 0;
    int lastButtonState = HIGH;

    // Inifinite Loop ------------------------------------------------------------
    while(1){

        int buttonState = GPIO_read(BUTTON_PIN);
        sevensegment_decoder(cnt % 10);
        if(buttonState == LOW && lastButtonState == HIGH) {
            cnt++;
            if (cnt > 9) cnt = 0;
        }
        lastButtonState = buttonState;
    }
}

void setup(void)
    {
    RCC_HSI_init();
    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, PULL_UP);

    sevensegment_decoder_init();
    uint16_t pins[] = {PA_5, PA_6, PA_7, PB_6, PC_7, PA_9, PA_8, PB_10};
    for (int i = 0; i < 8; i++) {
        GPIO_otype(pins[i], PUSH_PULL);
        GPIO_pupd(pins[i], NO_PUPD);
        GPIO_ospeed(pins[i], MEDIUM_SPEED);
    }
}
```

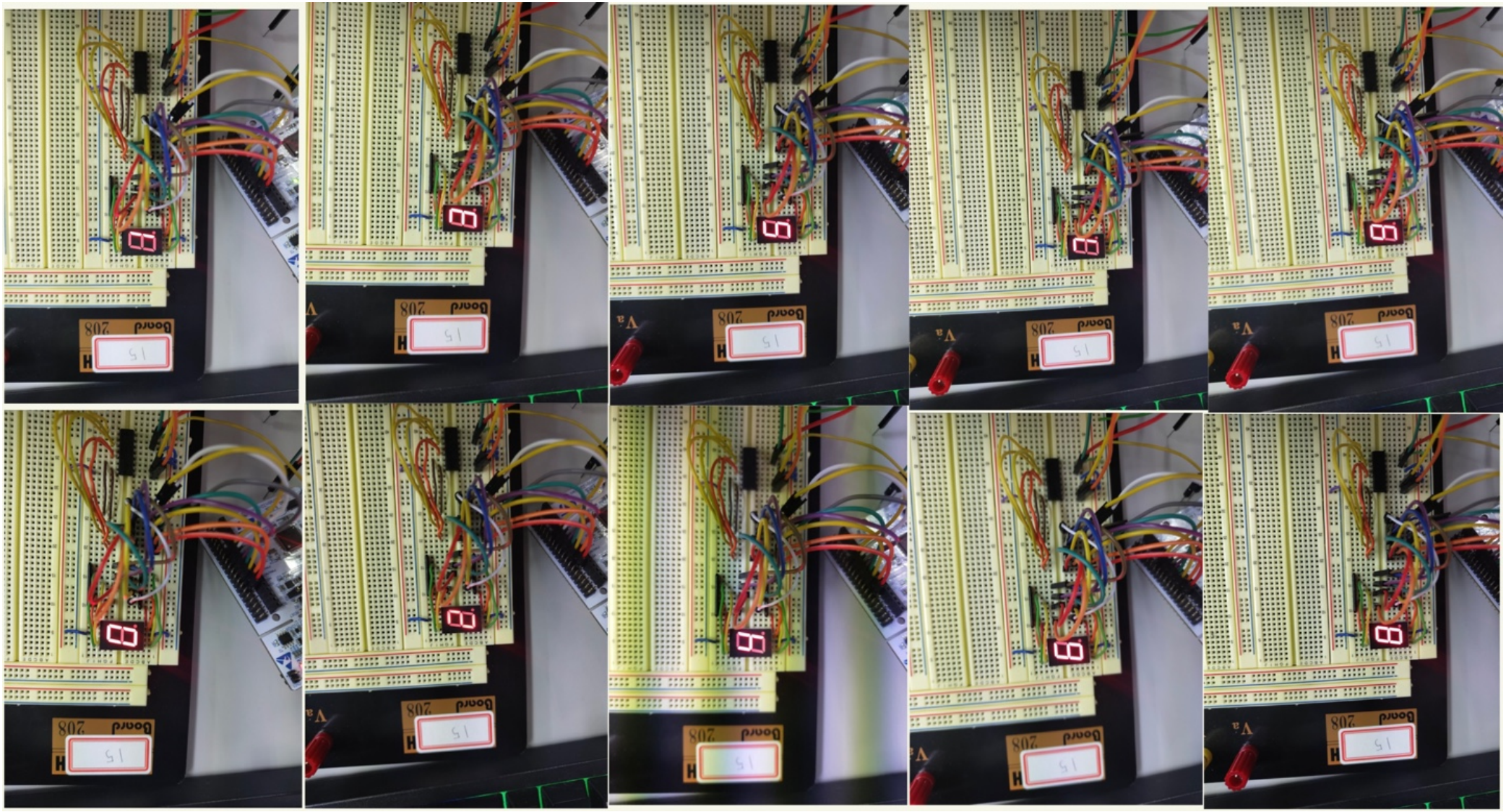
## Connection Diagram

Circuit diagram



# Results

Experiment images and results


15
Board 208

demo video link

## Discussion

Without using a decoder, 7 segments of LEDs were assigned to the MCU pin one-to-one to operate. If the pins of the 7 segment are organized well with bits and well defined at array, not only numbers from 0 to 9 but also alphabets can be expressed

---

# Reference

> 1. 7segment 원리와 구성

---

# Troubleshooting

[-]